

Sreeraaghav Raja
4/26/26
40990068

EE Design 1 Technical Report

Final Project

Luminous Library

Introduction

Luminous Library was the brainchild of an individual who was driven to do an IoT project but did not have the time to properly implement the IoT part of the project. The purpose of this project was to embrace my passion for reading and solve an issue that I've had in the past: logging my reading progress in a gamified way. This project serves as a high-fidelity prototype of a book logger that integrates both analog inputs and outputs (DACs, potentiometers, Photoresistors) and digital inputs and outputs (switches, buttons, LEDs) to implement a functional book logger design. The prototype implements visual and auditory feedback for the logging of a book upon a completion stage. There are also 2 additional modes where one recommends a book for each of the 16 basic genres and another that adds 16 new genres to log.

Methods

Background Information

The necessary information required for this project lies more in software than in hardware. Nevertheless, I will first explain the analog input sensors in depth before I explain anything related to the software. The project also utilized an array of digital inputs and outputs, but they were much simpler as they were switches, buttons, and LEDs that did not use Pulse-Width Modulation (PWM) therefore they will be briefly explained after the analog components.

Analog Input Sensor: Potentiometer

The two analog input sensors utilized in this project are the potentiometer and the photoresistor. The potentiometer is an incredibly unique analog component because of the way it incorporates mechanical engineering



Figure 1: Potentiometer Internal Design

elements into its design. The potentiometer is

Analog Input Sensor: Photoresistor

Analog Output: DAC

Digital Input: Switches and Buttons

Digital Output: LEDs

Software: FreeRTOS

Cyber-Informed Engineering Best Practice

Design Steps:

Software Block Diagram

Hardware Block Diagram

Altium Schematic

Altium Board Layout

BOM

Include:

- Discussion of analog input sensor (e.g., temp sensor, photoresistor, potentiometer, etc.) and how the sensor works, and any equations used (provide references). I want to see a little research on the physics behind the sensor; there is plenty mechanical physics behind the potentiometer.
- Discussion of design for Cyber-Informed Engineering best practice (refer to Dr. Forte's references in discussion assignment)
- Software block diagram (with detailed discussion)
- Hardware block diagram (with detailed discussion)
- Altium schematic (with detailed discussion)
- Altium board layout (with detailed discussion)
- Bill of materials with cost of components and total cost (with some discussion about commercialization or comparison to similar commercial consumer electronics)

Results

The results should be displayed in this section, which include figures and descriptive text for each figure. Figure captions need to be given as well, occurring below the figure. Use subheadings as necessary.

Include:

- Pictures of the final PCB
- Documentation of the system function
 - Pictures of LCD output
 - Oscilloscope measurement of analog signal taken at the speaker terminals and DAC output – if different, explain
 - Anything else you want to include to show off your hard work

PCB Pictures

Oscilloscope Output

Discussion/Conclusion

The conclusion should include a short synopsis of the use cases/ goals of your system and discussion of trials encountered and lessons learned. If hardware or software functionality was not perfect explain how it could be improved.

References

Put the references in any standard citation form here. Websites are allowed.

Appendix

Header Files:

FreeRTOSConfig.h

```
#ifndef FREERTOS_CONFIG_H
#define FREERTOS_CONFIG_H

/* Hardware Specific Settings for RP2350 */
#define configCPU_CLOCK_HZ            ( 150000000UL )
#define configTICK_RATE_HZ            ( ( TickType_t ) 1000 )
#define configMAX_PRIORITIES          ( 5 )
#define configMINIMAL_STACK_SIZE      ( 256 )

/* RP2350 Specific Port Settings - REQUIRED FOR CORTEX-M33 */
#define configENABLE_FPU              1
#define configENABLE_MPU              0
#define configENABLE_TRUSTZONE        0
#define configRUN_FREERTOS_SECURE_ONLY 1
#define configMAX_API_CALL_INTERRUPT_PRIORITY 16
#define configMAX_SYSCALL_INTERRUPT_PRIORITY 16

/* SMP / Multi-core Settings */
#define configNUMBER_OF_CORES         2
#define configUSE_PASSIVE_IDLE_HOOK    0
#define configTICK_CORE               0
#define configRUN_MULTIPLE_PRIORITIES  1
#define configUSE_CORE_AFFINITY        1
/* portCRITICAL_NESTING_IN_TCB removed - port defines this as 0 */
#define configUSE_TASK_PREEMPTION_DISABLE 0

/* SMP Spinlock configuration */
#define configSMP_SPINLOCK_0          16
#define configSMP_SPINLOCK_1          17

/* Memory Management */
#define configSUPPORT_DYNAMIC_ALLOCATION 1
#define configTOTAL_HEAP_SIZE          ( 64 * 1024 )
#define configTIMER_TASK_STACK_DEPTH   ( configMINIMAL_STACK_SIZE * 2 )

/* Task Scheduling */
#define configUSE_PREEMPTION            1
#define configUSE_TIME_SLICING         1
#define configUSE_IDLE_HOOK            0
#define configUSE_TICK_HOOK            0
```

```

#define configUSE_MUTEXES 1
#define configUSE_RECURSIVE_MUTEXES 1
#define configUSE_COUNTING_SEMAPHORES 1

/* Software Timers */
#define configUSE_TIMERS 1
#define configTIMER_TASK_PRIORITY ( 3 )
#define configTIMER_QUEUE_LENGTH ( 10 )

/* API Function Inclusion */
#define INCLUDE_vTaskPrioritySet 1
#define INCLUDE_vTaskDelay 1
#define INCLUDE_vTaskDelete 1
#define INCLUDE_vTaskSuspend 1
#define INCLUDE_xQueueGetMutexHolder 1
#define INCLUDE_xTimerPendFunctionCall 1

/* Misc */
#define configALLOW_MUTEX_SELF_RECURSION 1
#define configUSE_16_BIT_TICKS 0
#define configIDLE_SHOULD_YIELD 1

#define configASSERT( x ) if( ( x ) == 0 ) { portDISABLE_INTERRUPTS(); for( ;; ); }

#endif /* FREERTOS_CONFIG_H */

```

Tasks.h

```

#ifndef _TASKS_H
#define _TASKS_H

#include <stdio.h>
#include "pico/stdlib.h"
#include "FreeRTOSConfig.h"
#include "FreeRTOS.h"

#define MAX_MAP_SIZE 256

// make a struct for the stored data
typedef struct{
    char genre[16];
    int progress;
} LibraryLog;

```

```

// initialize a list of all the data;
extern LibraryLog DataMap[MAX_MAP_SIZE];

void Progress_Task(void *pvParameters);
void button_isr(uint gpio, uint32_t events);
void Button_Task(void *pvParameters);
void Switch_Task(void *pvParameters);
void Chime_Task(void *pvParameters);
void Photo_Task(void *pvParameters);
void Log_Task(void *pvParameters);

extern TaskHandle_t xProgressHandle;
extern TaskHandle_t xButtonHandle;
extern TaskHandle_t xSwitchHandle;

#endif /* _TASKS_H */

```

Semaphores.h

```

#ifndef _SEMAPHORES_H
#define _SEMAPHORES_H

#include "FreeRTOS.h"
#include "pico/stdlib.h"
#include "semphr.h"

extern SemaphoreHandle_t xButtonSemaphore;
extern SemaphoreHandle_t xLedMutex;
extern SemaphoreHandle_t xChimeSemaphore;
extern SemaphoreHandle_t xADCMutex;

#endif /* _SEMAPHORES_H */

```

LCD_Init.h

```

#ifndef _LCD_INIT_H
#define _LCD_INIT_H

#include "pico/stdio.h"
#include "hardware/i2c.h"

```

```

#define I2C_PORT i2c0
#define I2C_SDA 4
#define I2C_SCL 5
#define LCD_ADDR 0x27 // Default Slave Address of the PCF8574T

void lcd_init(void);
void lcd_clear(void);
void lcd_set_cursor(uint8_t row, uint8_t col);
void lcd_print(const char *str);
void lcd_command(uint8_t cmd);
void lcd_data(uint8_t data);void lcd_init(void);

#endif /* _LCD_INIT_H */

```

Components.h

```

#ifndef _COMPONENTS_H
#define _COMPONENTS_H

#include <pico/stdlib.h>

#define DEBOUNCE_TIME 100

#define SWITCH1_PIN 0
#define SWITCH2_PIN 1
#define SWITCH3_PIN 2
#define SWITCH4_PIN 3
#define LOG_SWITCH 6

#define LED1_PIN 9
#define LED2_PIN 10
#define LED3_PIN 11
#define LED4_PIN 12
#define LED5_PIN 13

#define BUTTON_PIN 15

void set_switch(uint8_t switch_pin);

```

```

void initialize_leds(uint8_t led_pin);
void toggle_leds(uint8_t led_pin);

#endif /*_COMPONENTS_H*/

```

Chime.h

```

#ifndef _CHIME_H
#define _CHIME_H

#include <pico/stdio.h>
#include "hardware/spi.h"
#define SAMPLE_COUNT 64
#define SPI_PORT spi0
#define SPI_CS      17
#define SPI_SCK      18
#define SPI_MOSI     19

// sample_rate_hz = desired_frequency * SAMPLE_COUNT
// e.g. 440 Hz tone -> 440 * 256 = 112640
#define CHIME_SAMPLE_RATE (440 * SAMPLE_COUNT)

extern uint8_t sine_table[SAMPLE_COUNT * 2];

void generate_sine(void);
void spi_start(void);
void chime_start(int sample_rate_hz);
void chime_stop(void);

#endif /*_CHIME_H*/

```

ADC_Init.h

```

#ifndef _ADC_INIT_H
#define _ADC_INIT_H

#include "hardware/adc.h"

#define POT_PIN 27 // for ADC1
#define POT_ADC 1
#define PHO_PIN 26 // for ADC0
#define PHO_ADC 0
#define ADC_CONV_FACTOR (3.3f/((1 << 12) - 1))

```

```
// initialize the adc
void adc_start(void);

#endif /*_ADC_INIT_H*/
```

Main_Init.h

```
#ifndef _MAIN_INIT_H
#define _MAIN_INIT_H

#include "adc_init.h"
#include "components.h"

void main_init(void);

#endif /*_MAIN_INIT_H*/
```

Source Files:

Tasks.c

```
#include <stdio.h>
#include <stdlib.h>
#include "pico/stdlib.h"
#include "FreeRTOSConfig.h"
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"
#include "../main_init.h"
#include "../semaphores.h"
#include "../lcd_init.h"
#include "math.h"
#include "../chime.h"
#include "../tasks.h"

TaskHandle_t xProgressHandle = NULL;
TaskHandle_t xButtonHandle = NULL;
TaskHandle_t xSwitchHandle = NULL;

// ----- State Flags ----- //
volatile bool start_flag = 0; // when program starts
```

```

volatile bool progress_flag    = 0; // for the progress bar
volatile bool genre_flag      = 0; // for genre chooing
volatile bool book_rec_flag   = 0; // dark mode: book recommendation
volatile int  log_inx         = 0; // index to track current location in
datamap
volatile bool log_mode        = false; // suppress chime while browsing logs

// ----- Photo State ----- //
typedef enum { PHOTO_NORMAL, PHOTO_DARK, PHOTO_BRIGHT } photo_mode_t;
volatile photo_mode_t photo_mode    = PHOTO_NORMAL;
volatile bool          photo_enabled = true;

// ----- Log into MAP ----- //
char current_genre[16] = "Mystery";
int current_progress   = 0;
LibraryLog DataMap[MAX_MAP_SIZE];

// ----- LUTs ----- //

// Shared genre names – used by normal mode and dark (book rec) mode
static const char *genres[16] = {
    "Mystery",      "Romance",      "Horror",      "Fantasy",
    "Psychological", "Sci-Fi",      "Classics",    "Historical",
    "Comedy",       "Philosophy",   "Classics",    "Biography",
    "Adventure",    "Manga/Comics", "True Crime",  "LGBTQ+",
};

static const char *book_recs[16] = {
    "Gone Girl",      // Mystery
    "Pride&Prejudice", // Romance
    "Pet Sematary",   // Horror
    "Mistborn",       // Fantasy
    "Fight Club",     // Psychological
    "Dune",            // Sci-Fi
    "1984",            // Classics
    "All Quiet...",   // Historical
    "Good Omens",     // Comedy
    "Fear&Trembling",  // Philosophy
    "Brave New World", // Classics
    "Night",           // Biography
    "Into the Wild",   // Adventure
    "Frieren",         // Manga/Comics
    "In Cold Blood",   // True Crime
    "Heated Rivalry",  // LGBTQ+
};

```

```

// 15 new genres for bright mode
static const char *bright_genres[16] = {
    "Satire",
    "Thriller",
    "Self-Help",
    "Poetry",
    "Drama",
    "Western",
    "Dystopian",
    "Mythology",
    "Military",
    "Political",
    "Travel",
    "Art & Design",
    "Music",
    "Sports",
    "Cooking",
    "Nature",
};

// ----- Tasks ----- //
void Progress_Task(void *pvParameters) {
    bool rst = 0;
    int prev_leds_on = 0;
    char buf[16];
    for(;;) {
        if(progress_flag){
            xSemaphoreTake(xADCMutex, portMAX_DELAY);
            adc_select_input(POT_ADC);
            uint16_t raw = adc_read();
            xSemaphoreGive(xADCMutex);

            printf("Raw Value: 0x%03x, voltage = %f V\n", raw, raw *
ADC_CONV_FACTOR);
            float voltage_adc = raw * ADC_CONV_FACTOR;
            float threshold = voltage_adc / 3.3f;
            int leds_on = (int)roundf(threshold * 5);

            xSemaphoreTake(xLedMutex, portMAX_DELAY);
            for(int i = LED1_PIN; i < LED1_PIN + 5; i++){
                if((i - LED1_PIN) < leds_on){ gpio_put(i, 1); }
                else{ gpio_put(i, 0); }
            }
            xSemaphoreGive(xLedMutex);
        }
    }
}

```

```

        if(abs(prev_leds_on - leds_on) >= 1){
            int percent = leds_on * 20;
            snprintf(buf, sizeof(buf), "Progress: %d%%", percent);
            current_progress = percent;
            lcd_clear();
            lcd_print(buf);
        }
        prev_leds_on = leds_on;
        rst = 1;
    }
    else{
        if(rst){
            lcd_clear();
            lcd_print("Registration");
            lcd_set_cursor(1, 0);
            lcd_print("Complete!");
            rst = 0;
        }
    }
    vTaskDelay(pdMS_TO_TICKS(100));
}

}

void button_isr(uint gpio, uint32_t events){
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    xSemaphoreGiveFromISR(xButtonSemaphore, &xHigherPriorityTaskWoken);
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}

void Button_Task(void *pvParameters){
    for(;;){
        if(xSemaphoreTake(xButtonSemaphore, portMAX_DELAY) == pdTRUE){
            vTaskDelay(pdMS_TO_TICKS(100)); // debounce
            if(!gpio_get(BUTTON_PIN) && !log_mode){
                if(!start_flag){
                    // --- Entry: choose path based on light condition ---
                    start_flag = 1;
                    photo_enabled = false;
                    if(photo_mode == PHOTO_DARK){
                        book_rec_flag = 1;
                    }
                    // PHOTO_BRIGHT and PHOTO_NORMAL both use the normal
                    progress/genre flow
                }
            }
        }
    }
}

```

```

        // photo_mode stays frozen (photo_enabled=false) so
Switch_Task can check it
    }
    else{
        // --- Exit / advance: depends on active state ---
        if(book_rec_flag){
            // one button press exits book rec state
            book_rec_flag = 0;
            start_flag    = 0;
            photo_enabled = true;
        }
        else{
            // Normal mode state machine: genre → progress → done
            if(!genre_flag && !progress_flag){
                genre_flag = 1;
            }
            else if(genre_flag && !progress_flag){
                genre_flag = 0;
                progress_flag = 1;
            }
            else{
                progress_flag = 0;
                start_flag    = 0;
                photo_enabled = true;
                DataMap[log_inx].progress = current_progress;
                snprintf(DataMap[log_inx].genre, 16, "%s",
current_genre);

                log_inx = (log_inx + 1) % MAX_MAP_SIZE;
                xSemaphoreGive(xChimeSemaphore);
            }
        }
    }
}

xSemaphoreTake(xLedMutex, portMAX_DELAY);
for(int i = LED1_PIN; i < LED1_PIN + 5; i++) gpio_put(i, 0);
xSemaphoreGive(xLedMutex);

while(xSemaphoreTake(xButtonSemaphore, 0) == pdTRUE);
}
}
}

void Switch_Task(void *pvParameters){
    uint8_t last_state = 0xFF; // used to only refresh LCD when necessary

```

```

uint8_t log_switch;

// different genres
typedef enum { SW_BOOK_REC, SW_GENRE, SW_IDLE } sw_mode_t;
sw_mode_t prev_mode = SW_IDLE;
bool use_bright = false;
const char* genre;

for(;;){
    uint8_t switches = 0;
    switches |= gpio_get(SWITCH1_PIN) << 0;
    switches |= gpio_get(SWITCH2_PIN) << 1;
    switches |= gpio_get(SWITCH3_PIN) << 2;
    switches |= gpio_get(SWITCH4_PIN) << 3;

    // Determine current mode
    sw_mode_t cur_mode;
    if(book_rec_flag) cur_mode = SW_BOOK_REC;
    else if(genre_flag) cur_mode = SW_GENRE;
    else
        cur_mode = SW_IDLE;

    // Reset last_state on mode transition so display refreshes immediately
    if(cur_mode != prev_mode){
        last_state = 0xFF;
        if(prev_mode == SW_BOOK_REC){
            lcd_clear();
            lcd_print("Luminous Library");
        }
        if(cur_mode == SW_GENRE){
            // Snapshot photo_mode once so bright vs normal stays stable
            use_bright = (photo_mode == PHOTO_BRIGHT);
        }
        prev_mode = cur_mode;
    }

    if(cur_mode == SW_BOOK_REC){
        if(switches != last_state){
            last_state = switches;
            lcd_clear();
            lcd_print(genres[switches]);
            lcd_set_cursor(1, 0);
            lcd_print(book_recs[switches]);
        }
    }
    else if(cur_mode == SW_GENRE){

```

```

        if(switches != last_state){
            last_state = switches;
            lcd_clear();
            genre = use_bright ? bright_genres[switches] : genres[switches];
            lcd_print(genre);
            snprintf(current_genre, 16, "%s", genre);
        }
    }
    else{ // SW_IDLE
    }

    vTaskDelay(pdMS_TO_TICKS(50));
}

}

void Chime_Task(void *pvParameters){
    for(;;){
        if(xSemaphoreTake(xChimeSemaphore, portMAX_DELAY) == pdTRUE){
            if(!log_mode){
                chime_start(CHIME_SAMPLE_RATE);
                while(!start_flag && !log_mode){
                    vTaskDelay(pdMS_TO_TICKS(100));
                }
                chime_stop();
            }
        }
    }
}

void Photo_Task(void *pvParameters){
    for(;;){
        if(photo_enabled){
            xSemaphoreTake(xADCMutex, portMAX_DELAY);
            adc_select_input(PHO_ADC);
            uint16_t raw = adc_read();
            xSemaphoreGive(xADCMutex);

            float voltage = raw * ADC_CONV_FACTOR;
            printf("Photo: 0x%03x, %.3f V\n", raw, voltage);

            if(voltage >= 2.0f){
                photo_mode = PHOTO_DARK;
            } else if(voltage < 0.5f){
                photo_mode = PHOTO_BRIGHT;
            }
        }
    }
}

```

```

        } else {
            photo_mode = PHOTO_NORMAL;
        }
    }
    vTaskDelay(pdMS_TO_TICKS(200));
}

}

void Log_Task(void *pvParameters) {
    for (;;) {
        // Wait until GP6 is high
        if (gpio_get(LOG_SWITCH)) {
            vTaskDelay(pdMS_TO_TICKS(50)); // Debounce
            if (gpio_get(LOG_SWITCH)) {

                log_mode = true;
                chime_stop();

                printf("GET FRICKED!");

                if (log_inx == 0) {
                    lcd_clear();
                    lcd_print("No Logs Found");
                    vTaskDelay(pdMS_TO_TICKS(1500));
                } else {
                    int i = 0;
                    while (i < log_inx && gpio_get(LOG_SWITCH)) {
                        lcd_clear();
                        lcd_print(DataMap[i].genre);

                        lcd_set_cursor(1, 0);
                        char buf[16];
                        snprintf(buf, sizeof(buf), "Prog: %d%%",
DataMap[i].progress);
                        lcd_print(buf);

                        i++;
                        vTaskDelay(pdMS_TO_TICKS(1500));
                    }
                }

                // Wait for switch release before restoring display
                while(gpio_get(LOG_SWITCH)){
                    vTaskDelay(pdMS_TO_TICKS(50));
                }
            }
        }
    }
}

```

```

        }
        log_mode = false;
        lcd_clear();
        lcd_print("Luminous Library");
    }
}
// Check every 100ms so we don't waste CPU
vTaskDelay(pdMS_TO_TICKS(100));
}
}

```

Semaphores.c

```

#include "FreeRTOS.h"
#include "pico/stdlib.h"
#include "semphr.h"

SemaphoreHandle_t xButtonSemaphore;
SemaphoreHandle_t xLedMutex;
SemaphoreHandle_t xChimeSemaphore;
SemaphoreHandle_t xADCMutex;

```

LCD_Init.c

```

#include "pico/stdlib.h"
#include "../lcd_init.h"

// got this from claude, but I know how it works

static void lcd_write(uint8_t data){
    i2c_write_blocking(I2C_PORT, LCD_ADDR, &data, 1, false);
}

static void lcd_pulse_enable(uint8_t data){
    lcd_write(data | 0x04); // EN high
    sleep_us(1);
    lcd_write(data & ~0x04); // EN low
    sleep_us(50);
}

static void lcd_send_nibble(uint8_t nibble, uint8_t mode){
    uint8_t data = (nibble & 0xF0) | mode | 0x08; // 0x08 = backlight on
}

```

```

        lcd_pulse_enable(data);
    }

static void lcd_send_byte(uint8_t data, uint9_t mode){
    lcd_send_nibble(data & 0xF0, mode); // upper nibble
    lcd_send_nibble((data << 4) & 0xF0, mode); // lower nibble
}

void lcd_command(uint8_t cmd){
    lcd_send_byte(cmd, 0x00); // RS=0
}

void lcd_data(uint8_t data){
    lcd_send_byte(data, 0x01); // RS=1
}

void lcd_init(void){
    i2c_init(I2C_PORT, 100 * 1000);
    gpio_set_function(I2C_SDA, GPIO_FUNC_I2C);
    gpio_set_function(I2C_SCL, GPIO_FUNC_I2C);
    gpio_pull_up(I2C_SDA);
    gpio_pull_up(I2C_SCL);

    sleep_ms(50);

    // force 8-bit mode 3 times
    lcd_send_nibble(0x30, 0x00);
    sleep_ms(5);
    lcd_send_nibble(0x30, 0x00);
    sleep_us(150);
    lcd_send_nibble(0x30, 0x00);

    // switch to 4-bit mode
    lcd_send_nibble(0x20, 0x00);

    // function set: 4-bit, 2 lines, 5x8 font
    lcd_command(0x28);
    // display on, cursor off, blink off
    lcd_command(0x0C);
    // clear display
    lcd_command(0x01);
    sleep_ms(2);
    // entry mode: increment, no shift
    lcd_command(0x06);
}

```

```

void lcd_clear(void){
    lcd_command(0x01);
    sleep_ms(2);
}

void lcd_set_cursor(uint8_t row, uint8_t col){
    uint8_t offsets[] = {0x00, 0x40};
    lcd_command(0x80 | (offsets[row] + col));
}

void lcd_print(const char *str){
    while(*str){
        lcd_data(*str++);
    }
}

```

Components.c

```

#include "pico/stdlib.h"
#include "../components.h"

void set_switch(uint8_t switch_pin){
    // default direction is input
    gpio_init(switch_pin);

    // set pin to have internal pullup - unnecessary
    gpio_pull_up(switch_pin);
}

void initialize_leds(uint8_t led_pin){
    gpio_init(led_pin);
    gpio_set_dir(led_pin, GPIO_OUT);
}

void toggle_leds(uint8_t led_pin){
    gpio_xor_mask64(1u << led_pin);
}

```

Chime.c

```

#include <pico/stdio.h>
#include "../chime.h"
#include "hardware/gpio.h"
#include <pico/time.h>
#include "math.h"

#ifndef M_PI
#define M_PI 3.14159265358979323846f
#endif

uint8_t sine_table[SAMPLE_COUNT * 2];

static volatile int sample_index = 0;
static struct repeating_timer dac_timer;

static bool dac_sample_irq(struct repeating_timer *t){
    gpio_put(SPI_CS, 0);
    spi_write_blocking(SPI_PORT, &sine_table[sample_index * 2], 2);
    gpio_put(SPI_CS, 1);
    sample_index = (sample_index + 1) % SAMPLE_COUNT;
    return true;
}

void chime_start(int sample_rate_hz){
    sample_index = 0;
    int period_us = 1000000 / sample_rate_hz;
    add_repeating_timer_us(-period_us, dac_sample_irq, NULL, &dac_timer);
}

void chime_stop(void){
    cancel_repeating_timer(&dac_timer);
    gpio_put(SPI_CS, 1);
}

void spi_start(void){
    spi_init(SPI_PORT, 2000 * 1000);
    spi_set_format(SPI_PORT, 8, SPI_CPOL_0, SPI_CPHA_0, SPI_MSB_FIRST);
    // set all the used pins
    gpio_set_function(SPI_SCK, GPIO_FUNC_SPI);
    gpio_set_function(SPI_MOSI, GPIO_FUNC_SPI);
    gpio_init(SPI_CS);

    // set the CS to be an output
    gpio_set_dir(SPI_CS, GPIO_OUT);

```

```

    // CS idles as high
    gpio_put(SPI_CS, 1);
}

void generate_sine(void){
    for(int i = 0; i < SAMPLE_COUNT; i++){
        float s = sinf(2.0f * M_PI * i / SAMPLE_COUNT);
        uint16_t sample = (uint16_t)((s + 1.0f) / 2.0f * 1023.0f); // 10-bit
        // LTC1661: [A3 A2 A1 A0 | D9..D0 | X X]
        // 0xF000 = write+update both DACs, data in bits 11-2
        uint16_t word = 0xF000 | ((sample & 0x3FF) << 2);
        sine_table[i * 2]      = (word >> 8) & 0xFF;
        sine_table[i * 2 + 1] = word & 0xFF;
    }
}

```

ADC_Init.c

```

#include "hardware/adc.h"
#include "../adc_init.h"

void adc_start(void){
    adc_init();
    adc_gpio_init(POT_PIN);
    adc_gpio_init(PHO_PIN);
}

```

Main_Init.c

```

#include "../components.h"
#include "../adc_init.h"
#include "../main_init.h"
#include "../lcd_init.h"
#include "../chime.h"

void main_init(void){
    for(int i = LED1_PIN; i < LED1_PIN + 5; i++){initialize_leds(i);}
    for(int i = SWITCH1_PIN; i < SWITCH1_PIN + 4; i++){set_switch(i);}
    set_switch(LOG_SWITCH);
    set_switch(BUTTON_PIN);
    adc_start();
}

```

```
    lcd_init();  
    spi_start();  
    generate_sine();  
}
```

Grading scheme:

2 – Poor: Missing one or more sections and listed bullet-point items; poor formatting and/or writing

3 – Adequate: All sections and listed items accounted for, ok formatting, figures and description of methods/results/conclusion. May be brief or inaccurate on methods/results/conclusion.

4 – Mastered: All sections and listed items accounted for, good formatting, figures and description of methods/results/conclusion (includes references and descriptive text)

5 – Outstanding: All sections and listed items accounted for, great formatting, figures and description of methods/results/conclusion (includes references and descriptive text). Includes text that conveys a deep understanding of the electronics in the project.